React 18 — State Management & Custom Hooks

Source basis: React official documentation; React 18-compatible APIs

Learning goals

- Explain core concepts and when they are useful.
- Read and adapt code/query examples.
- Recognize trade-offs, common errors, and verification steps.

How to use this chapter

Read the concept first, run examples in an isolated environment, then use the official references at the end of the chapter to verify version-specific behavior. The examples are teaching samples, not production-ready systems.

1. Scope and state model

React state is data a component remembers between renders. State is local to a component instance unless lifted to a common parent or shared through Context or an external store. Updating state requests a render; it does not mutate the current render's value.

Start with local state, derive values where possible, lift shared state to the closest common owner, and introduce Context or a store when the sharing pattern warrants it.

Custom Hooks reuse stateful logic. They do not automatically share one state instance between callers; each call has its own state unless the Hook intentionally connects to shared state.

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

2. useState and functional updates

useState returns the current state and a setter. When the next value depends on the previous value, use a functional setter to compose queued updates correctly.

Calling a setter does not change the variable in the current function execution. Treat state as a render snapshot. For objects and arrays, create a new object or array instead of mutating existing state.

Example code / SQL

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  function addThree() {
    setCount(c => c + 1);
    setCount(c => c + 1);
    setCount(c => c + 1);
  }
  return <button onClick={addThree}>Count: {count}</button>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

3. State design and derived values

Store the minimum information needed to represent the UI. If a value can be calculated from props or state during render, derive it instead of storing a duplicate that can become inconsistent.

For example, store items and a filter, then derive visibleItems. Avoid storing items, filteredItems, and count separately if they all come from the same source.

Changing a component's key can intentionally reset state, but use this deliberately because it resets state below that identity.

Example code / SQL

```
import { useState } from "react";

const initial = [
  { id: 1, title: "Read docs", done: false },
  { id: 2, title: "Write test", done: true },
];

export default function TaskList() {
  const [tasks, setTasks] = useState(initial);
  const [showDone, setShowDone] = useState(false);
  const visible = tasks.filter(t => showDone ? t.done : !t.done);
  function toggle(id) {
    setTasks(old => old.map(t =>
      t.id === id ? { ...t, done: !t.done } : t));
  }
  return <div>
    <button onClick={() => setShowDone(v => !v)}>Toggle filter</button>
    {visible.map(t => <label key={t.id}>
      <input type="checkbox" checked={t.done}
        onChange={() => toggle(t.id)} />{t.title}
    </label>)}
  </div>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

4. Sharing state by lifting it up

When sibling components need coordinated state, move it to their closest common parent and pass values and handlers as props. This creates a single source of truth.

Controlled components receive current values through props and report changes through callbacks. Use them when a parent needs to coordinate, validate, or persist input.

Do not lift state higher than necessary. Keeping unrelated state local reduces coupling and unnecessary renders.

Example code / SQL

```
import { useState } from "react";

function TemperatureInput({ value, onChange }) {
  return <input value={value}
    onChange={e => onChange(e.target.value)} />;
}

export default function Converter() {
  const [celsius, setCelsius] = useState("20");
  const fahrenheit = Number(celsius) * 9 / 5 + 32;
  return <>
    <TemperatureInput value={celsius} onChange={setCelsius} />
    <p>Fahrenheit: {fahrenheit.toFixed(1)}</p>
  </>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

5. Reducers for complex transitions

useReducer is useful when transitions are related, involve multiple fields, or benefit from a centralized action model. A reducer receives current state and an action and returns the next state without mutating the current one.

Reducers do not automatically make state global. They can be local, shared through Context, or used with a library. Keep actions descriptive and reducers pure; network calls belong outside reducers.

Example code / SQL

```
import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "increment": return { count: state.count + 1 };
    case "decrement": return { count: state.count - 1 };
    case "reset": return { count: 0 };
    default: throw new Error("Unknown action");
  }
}

export default function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return <>
    <p>{state.count}</p>
    <button onClick={() => dispatch({ type: "increment" })}>+</button>
    <button onClick={() => dispatch({ type: "decrement" })}>–</button>
    <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
  </>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

6. Context for cross-tree values

Context provides a value to descendants without passing props through every intermediate layer. It can suit theme, locale, authenticated-user information, or reducer-backed state shared across a subtree.

Context is not a universal replacement for props or state libraries. When a provider value changes, consumers that read the context update. Keep values focused and split unrelated contexts when appropriate.

Context carries a value; combine it with useState or useReducer when descendants need to update shared state.

Example code / SQL

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext("light");
function ThemeLabel() {
  return <p>Theme: {useContext(ThemeContext)}</p>;
}
export default function App() {
  const [theme, setTheme] = useState("light");
  return <ThemeContext.Provider value={theme}>
    <ThemeLabel />
    <button onClick={() =>
      setTheme(t => t === "light" ? "dark" : "light")}>
      Toggle theme
    </button>
  </ThemeContext.Provider>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

7. Custom Hooks and reusable logic

A custom Hook is a JavaScript function whose name starts with use and that may call other Hooks. It extracts reusable stateful logic while leaving each component responsible for rendering.

Custom Hooks share logic, not automatically the same state. If two components call useCounter(), each has its own counter unless the Hook connects to shared state.

Keep Hooks focused on a concrete purpose such as useOnlineStatus, useWindowSize, or useTextField.

Example code / SQL

```
import { useEffect, useState } from "react";

export function useOnlineStatus() {
  const [online, setOnline] = useState(navigator.onLine);
  useEffect(() => {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    window.addEventListener("online", on);
    window.addEventListener("offline", off);
    return () => {
      window.removeEventListener("online", on);
      window.removeEventListener("offline", off);
    };
  }, []);
  return online;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

8. Custom Hook example: text fields

A small Hook can centralize repeated field behavior. Return a clear API and keep validation rules explicit. A generic Hook may manage value and reset, while domain-specific validation remains in the form.

For larger forms, a dedicated form library may be appropriate. React 18 Strict Mode can repeat development checks to expose unsafe side effects, so effects should clean up listeners and timers.

Example code / SQL

```
import { useState } from "react";

function useTextField(initialValue = "") {
  const [value, setValue] = useState(initialValue);
  return {
    value,
    onChange: e => setValue(e.target.value),
    reset: () => setValue(initialValue)
  };
}

export default function ProfileForm() {
  const name = useTextField("");
  const email = useTextField("");
  return <form onSubmit={e => {
    e.preventDefault();
    console.log({ name: name.value, email: email.value });
  }}>
    <input placeholder="Name" value={name.value} onChange={name.onChange} />
    <input type="email" placeholder="Email" value={email.value}
      onChange={email.onChange} />
    <button>Save</button>
  </form>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

9. Rules of Hooks, effects, and performance

Call Hooks at the top level of function components or custom Hooks, not inside loops, conditions, or nested functions. Stable call order is essential.

useEffect synchronizes with external systems such as listeners, timers, and connections. It is not the default place to calculate values that can be derived during render.

Optimize after observing a real issue. React.memo, useMemo, and useCallback have specific purposes but can add complexity. Start with sensible state placement, stable keys, and clear component boundaries.

Example code / SQL

```
import { useEffect, useState } from "react";

export default function Clock() {
  const [now, setNow] = useState(() => new Date());
  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id);
  }, []);
  return <time>{now.toLocaleTimeString()}</time>;
}
```

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

10. Testing, exercises, and recap

Test user-visible behavior: inputs update the UI; reducers return expected states; custom Hooks clean up listeners; context consumers respond to provider changes; and repeated clicks do not rely on stale state.

Exercises: implement a cart reducer; lift selected-tab state to a parent; write useOnlineStatus and verify cleanup; and replace duplicated derived state with a computed value.

Review questions: When should state be lifted? What does a custom Hook reuse? Why should duplicated derived state be avoided?

Implementation and review notes

Keep state minimal and immutable, obey the Rules of Hooks, and test observable behavior. Effects that subscribe to external systems should clean up correctly.

Official references and further reading

These trusted official sources are the authority for version-specific details. This PDF summarizes key ideas; consult the linked references for full API specifications and additional examples.

React: Managing State
https://react.dev/learn/managing-state

React: Sharing State Between Components
https://react.dev/learn/sharing-state-between-components

React: useState reference
https://react.dev/reference/react/useState

React: Reusing Logic with Custom Hooks
https://react.dev/learn/reusing-logic-with-custom-hooks

React: Rules of Hooks
https://react.dev/reference/rules-of-hooks

React 18 release blog
https://react.dev/blog/2022/03/29/react-v18

Suggested practice checklist

• Confirm the runtime/version.
• Run the example in an isolated project or disposable database.
• Test one normal case and one failure/edge case.
• Record commands, expected output, and any assumptions.